

Assignment Statement and Objective(s):

The objective of this assignment is to interface the Raspberry Pi with a DHT11 temperature and humidity sensor over a single-wire protocol. The various parts of the assignment explore low-level hardware control, polling-based and interrupt-based communication, checksum validation, and file logging in the C programming language.

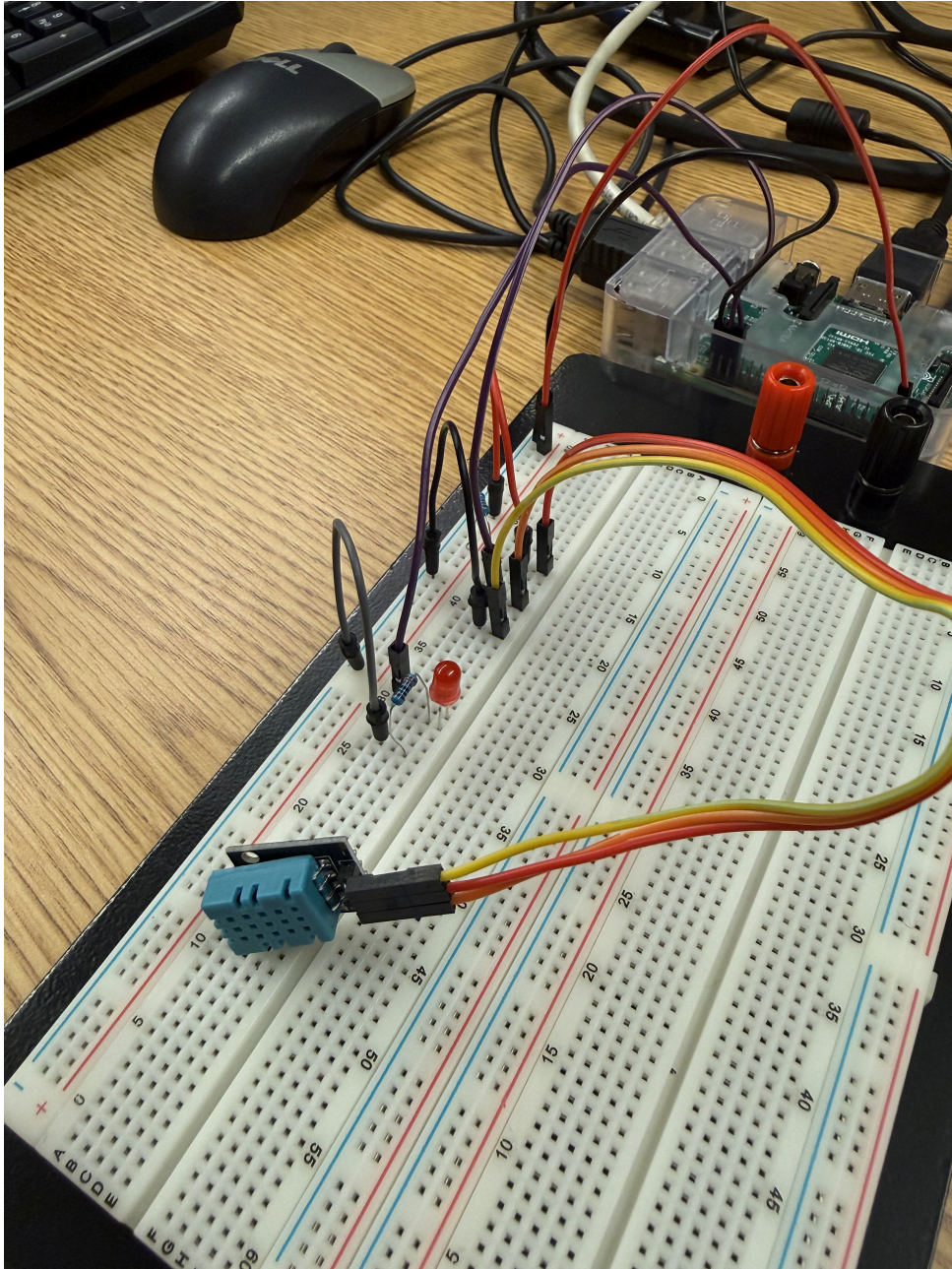
Summary of Contents

- **Part 1:** Implemented polling-based IO implementation. Busy-wait polling was employed to capture and decode 40 bits of data from the DHT11, validating the received data with a checksum.
- **Part 2:** Implemented interrupt-based IO implementation. Edge-triggered Interrupt Service Routine (ISR) was employed via a state machine backed by wiringPiISR.
- **Part 3:** Implemented LED actuation based on sensor readings. Used a circular buffer that calculates a moving average of the previous 10 temperature readings, turning on an LED when the average exceeds a preset threshold, and turning it off when the average falls below that threshold.

Methodology and Steps:**Hardware Configuration**

Communication with the DHT11 requires a bidirectional single-wire data line, power (3.3V) and ground. The data line must be connected to a GPIO pin on the pi as well as to a pull-up resistor of 10 kOhms (routed from the data pin to the 3.3V power rail). The LED actuator uses the LED circuit, where we used a small resistor (220 Ohms) and a red LED.

- DHT11 Data Pin: Connected to GPIO 25 (`#define DHT11_PIN 25`) and a 10 kOhm pull up resistor
 - The DHT11 features an S next to the data pin
- LED Indicator: Connected to GPIO 24 (`#define LED_PIN 24`), routed through a 220 Ohm resistor
- Ground and Power: 3.3V power routed to the positive rail or ground to the negative rail on the breadboard.
 - The DHT11 features a - next to the ground pin
 - The power pin on the DHT11 is directly adjacent to the ground pin



The pins explained and shown above were chosen following the output of `gpio readall`.

Part 1: Polling I/O Implementation

The Pi must manage both bus direction (flipping between input and output on a single pin) and timing pulse measurement (to decode received data based on the spec sheet):

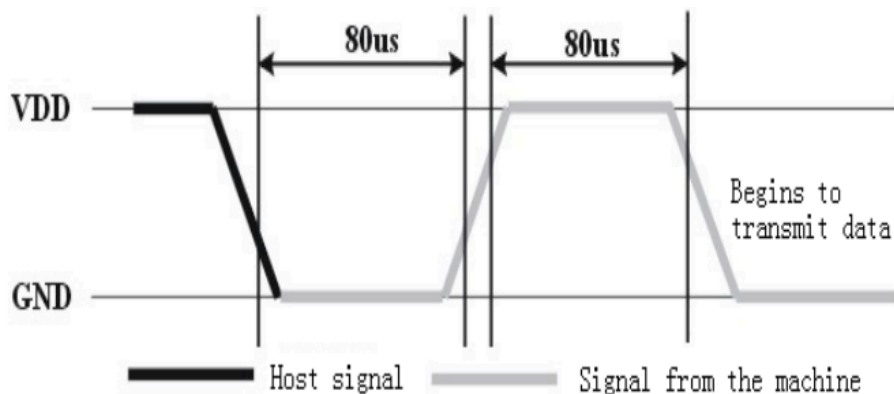
1. Host Sends Start Signal (`req_measure`)

- The Pi configures `DHT11_PIN` as `OUTPUT` and drives the line `LOW` for 18ms to signal the DHT11
- The Pi pulls the line high for 40us, then reconfigures the pin to `INPUT` to release the bus and wait for the DHT11's response (data)

2. Sensor Response

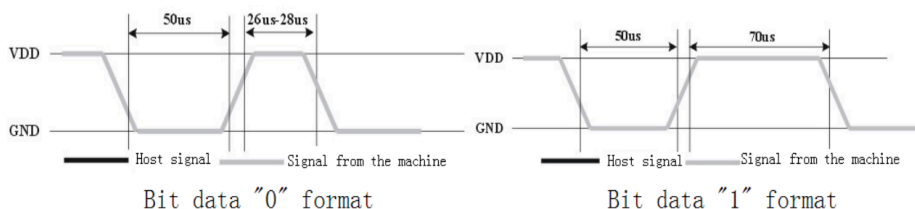
- The DHT11 responds by pulling the line low for about 80us, followed by high for 80us
- The DHT11 then sends 40 bits of data (5 bytes)
 - Byte 1: Relative Humidity Integer

- Byte 2: Relative Humidity Decimal
- Byte 3: Relative Temperature Integer
- Byte 4: Relative Temperature Decimal
- Byte 5: Checksum



3. Polling & Pulse Discrimination

- A polling loop samples the pin's state and waits for it to change to the next state
- Each data bit begins with a 50us LOW level, followed by a variable width (based on data state) HIGH level:
 - Bit '0': 26-28us HIGH
 - Bit '1': 70us HIGH
- A midpoint threshold of 45us is used (`#define PULSE_WIDTH_THRESHOLD_US 45`), meaning that any counter durations over 45us are considered 1, otherwise 0



4. Data Verification & File Logging

- The checksum is verified by adding the first 4 bits and comparing it to the actual 5th bit: $B0 + B1 + B2 + B3 == B4$
 - Since the data are represented as bytes (`uint8_t`), they naturally wrap and the checksum is thus in modulo 256 arithmetic
- When valid, values are decoded into a struct (`Data`) and logged with a date and time to both the stdout stream (terminal) and a log file (`temper_poll.log`)

Part 2: Interrupt I/O Implementation

1. ISR Registration

- Configure the ISR interface using `wiringPiISR` on the rising edge (`INT_EDGE_RISING`)
 - The arguments to this function are the pin, edge, and callback function
 - The function is thus called with: `wiringPiISR(sensor_pin, INT_EDGE_RISING, sensor_read_isr)`

2. State Machine

- `INIT_PULL_LINE_LOW`: The Pi asserts the 18ms start pulse and transitions the line to INPUT with pull-up configuration
- `INPUT_JUST_ENABLED & HIGH_ACK`: Absorbs the initial pulses from the sensor (see previous diagrams)
- `BIT_READ_RISING`: On every rising edge, the ISR records timestamps using the `micros` function. The pulse width is then calculated by subtracting the previous rising edge time from the current time, minus the pre-bit delay (50us). Discrimination of the pulse is decoded with a 10us buffer
 - If the delta is less than 28us + 10us, the bit is decoded as a zero
 - If the delta is less than 70 + 10us, the bit is decoded as a one
 - Otherwise, the state machine moves to an error state since no data was recorded
- `READ_COMPLETE`: Once all 40 bits are recorded, the bytes are extracted from the byte array via `extract_byte_at_offset`
 - The bytes are checksummed following the procedure described in Part 1

3. File Logging

- All readings are recorded to stdout and a file (`temper_interrupt.log`)

4. Mitigating Linux Non-Preemption

- Both this part and part 1 use `try_set_prio(99)` to try and raise the execution to real time round robin scheduling
 - This is done to attempt to mitigate the effect of interruptions from the linux scheduler

Part 3: LED Actuator over Moving Temperature Average

1. Circular Buffer Smoothing

- Readings will be gathered using Part 1's polling approach due to inconsistencies perceived in the interrupt-based method
- The integer and decimal parts of the data are combined into a floating point value via `parse_from_parts(dec_part, frac_part)`
- Parsed samples are pushed into a statically allocated 10 element floating point buffer (size determined at compile time via `#define MAXIMUM_TEMPERATURE_READINGS 10`)
- `average_celsius()` returns the rolling average of the current buffer, reducing the impact of an erroneous reading

2. Threshold Handling

- The sample loop polls every second (`#define LOOP_TIMEOUT_MS 1000`), consistent with parts 1 and 2
- If the current average temperature exceeds the set threshold (`#define TEMPERATURE_THRESHOLD_CELSIUS 26.0`):
 - `digitalWrite(LED_PIN, HIGH)` turns the red LED on
- Otherwise, `digitalWrite(LED_PIN, LOW)` turns the LED off
- All temperature readings are recorded to stdout and a file (`temper_threshold.log`)

3. Testing Procedure

- We warmed up the sensor by vigorously rubbing our hands together and smothering the sensor immediately after
- Sensor values were monitored until the temperature exceeded the threshold and the LED turned on
- We then removed our hands from the sensor, waited for the temperature to drop below the threshold, and confirmed that the LED turned off

Results and Observations:

Part 1 Results

The polling logic reliably decoded data received by the DHT11 when elevated to real-time priority.

Snippet from Log File

```
[2026-09-22 17:16:13] Temp: 22.6 C, Humidity: 47.0 %, Checksum: 4B  
[2026-09-22 17:16:14] Failed to read DHT11 data
```

The log shows two entries, one where the data was successfully read (checksum verified) and one where the data could not be read to completion (checksum invalid). The checksum show is validated by summing the data components and checking with the final byte of data provided by the sensor ($22 + 6 + 47 + 0 == 0x4B$). The linux scheduler resulted in inconsistent delays, likely causing the data drops.

Part 2 Results

The interrupt-based implementation also successfully captured data, but experienced some noticeable differences with Part 1.

Comparison with Polling

Metric	Polling	Interrupt
CPU Utilization	High during reads due to spinning on the delay counter	Low as the CPU can be idle until a rising edge fires
Sensitivity	Count will overflow if preempted	Sensitive to latency in response to interrupts
Complexity	Very simple linear loop	Complex state machine requiring more boilerplate and timestamp tracking
Priority	Benefits from real time round robin sched	Real time is necessary to prevent dropped/misclassified bits

Snippet from Log File

```
[2026-09-22 16:59:21] Temp: 25.3 C, Humidity: 54.1 %, Checksum: 53  
[2026-09-22 16:59:22] Failed to read DHT11 data
```

The log shows two entries, one where the data was successfully read (checksum verified) and one where the data could not be read to completion (checksum invalid). The higher temperature of the reading comes from holding the sensor in our hands at the time of measurement. Interrupt callback overhead likely resulted in failed readings.

Part 3 Results

During baseline testing, the room temperature averaged 22C, keeping the LED in an off state. When we smothered the sensor with our warm hands, the temperature readings rose gradually over about 30 seconds to around 27C, triggering the LED's threshold. We then removed our hands from the sensor and waited for the readings to dip down below the threshold.

The amount of time it took for the sensor to respond to readings was surprising to us. We believe there is something internal in the sensor that takes a while to respond to environmental stimuli, and that this response is even slower when cooling down. This makes sense as the sensor is made of plastic which is not the most efficient at heat transfer.

Snippet from Log File

```
[2026-09-22 18:08:30] Temperature is below the threshold of 26.000000 C: 22.600000 C  
[2026-09-22 18:09:40] Temperature is below the threshold of 26.000000 C: 25.630001 C  
[2026-09-22 18:09:41] Temperature exceeded the threshold of 26.000000 C: 26.070002 C  
[2026-09-22 18:09:42] Failed to read DHT11 data  
[2026-09-22 18:11:15] Temperature is below the threshold of 26.000000 C: 26.000000 C
```

The five entries from this log, in order, show: A baseline reading of the room, a reading taken as the sensor was warming up to the heat of our hands, a reading where the sensor's average temperature has surpassed the set threshold of 26C (turning on the LED), one where the checksum could not be verified, and one where the average temperature dropped to the set threshold (turning off the LED since the comparison was open at the bound). Since this code uses the polling based implementation, it suffers from the same scheduling drawback as discussed in Part 1.

Conclusions:

Techniques Learned

- Gained practical experience with implementing handshake protocols
- Observed the real constraints of attempting to execute very granular interrupt routines in a non real-time environment. Polling, while more expensive in general, can be less susceptible to scheduler interrupts than an interrupt-based solution when running at such small delays
- Learned one of the ways data can be transmitted to embedded devices and verified using checksums and bitwise operations
- Applied a moving average over a continuous stream of discrete sensor readings to display changes in sensor readings as LED state changes

Thoughts on the Assignment

- The DHT11 sensors are very inconsistent and take a long time to respond to changes in the environment. This made testing slow, though we understand that the cost of better sensors is a limiting factor here
- The transition between polling and interrupt based approaches was helpful for understanding nuances in real time scheduling on linux devices
- The diagrams provided on canvas were often hard to decipher given the mix of English and non-English instructions

Notes:

-